A Brief Introduction to Geometric Deep Learning

AI for complex data

Jason McEwen | 25 July 2022 | TDS Archive (Medium)

Deep learning is hard. While universal approximation theorems show that sufficiently complex neural networks can in principle approximate “anything”, there is no guarantee that we can find good models.

Great progress in deep learning has nevertheless been made by judicious choice of model architectures. These model architectures encode inductive biases to give the model a helping hand. One of the most powerful inductive biases is to leverage notions of geometry, giving rise to the field of geometric deep learning.

The term geometric deep learning was first coined by Michael Bronstein, a pioneer of the field (see his posts for interesting insights on a lot of the latest deep learning research, as well as extensive overviews of the field). In this post, rather then getting deep into the technical weeds, we present a very brief introduction to geometric deep learning. We largely follow the excellent recent book by Bronstein and colleagues [1] but provide our own unique take, and focus on high level concepts rather than technical details.

Geometric Priors

Fundamentally, geometric deep learning invovles encoding a geometric understanding of data as an inductive bias in deep learning models to give them a helping hand.

Our geometric understanding of the world is typically encoded through three types of geometric priors:

1. Symmetry and invariance

2. Stability

3. Multiscale representations

One of the most common geometric priors is to encode symmetries and invariances to different types of transformations. In physics, symmetries are typically expressed by the invariance of physical systems under transformations. If we know the real world exhibits certain symmetries, then it makes sense to encode those symmetries directly into our deep learning model. That way we are able to give the model a helping hard

so that is does not have to learn the symmetry but in some sense already knows it. Harnessing symmetry in deep learning is elaborated further in our previous article on What Einstein Can Teach Us About Machine Learning.

As an example of encoding symmetries and invariances, traditional convolutional neural networks (CNNs) exhibit what is called translational equivariance, as illustrated in the diagram below of a cat's face. Consider the model's feature space (on the right). If the camera or cat moves, i.e. is translated in the image, content in

the feature space should more similarly, i.e. is also translated. This property is called translational equivariance and in a sense ensures a pattern (cat's face) need only be learnt once. Rather than having to learn the pattern in all possible locations, by encoding translational equivariance in the model itself we ensure the pattern can then be recognised in all locations.

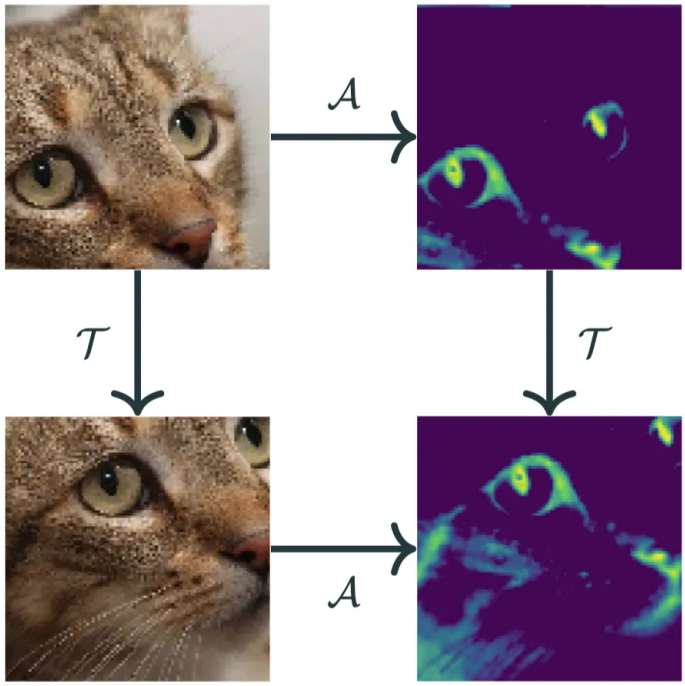


Figure 1. Translational equivariance: an image and its feature representation under translation.

Another common geometric prior is to ensure stability of the representation space. We can consider differences between data instances as due to some distortion that would map one data instance into another. For a classification problem, for

example, small distortions are responsible for variations within a class, whereas larger distortions can map data instances from one class to another. The size of the distortion between two data instances then captures how "close", or similar, one data instance is to another. For a representation space to be well-behavied and support effective deep learning, we should preserve measures of similarlity between data instances. To preserve similarity in the representation space, feature mappings must exhibit a form a stability.

As a representative example consider the classification of hand-written digits. The original image space and its representation space are illustrated in the following diagram. Small distortions map one 6 into another, capturing intra-class variations between different instances of a hand drawn 6. In the representational space, these data instances should remain close. Larger distortions, however, can map a 6 into an 8, capturing inter-class variations. Again, in the representation space the measure of similarity should be preserved and so there should be a larger separation between 6s and 8s in the representation space. Stability of the feature mapping is required to ensure such distances are preserved to facilitate effective learning.

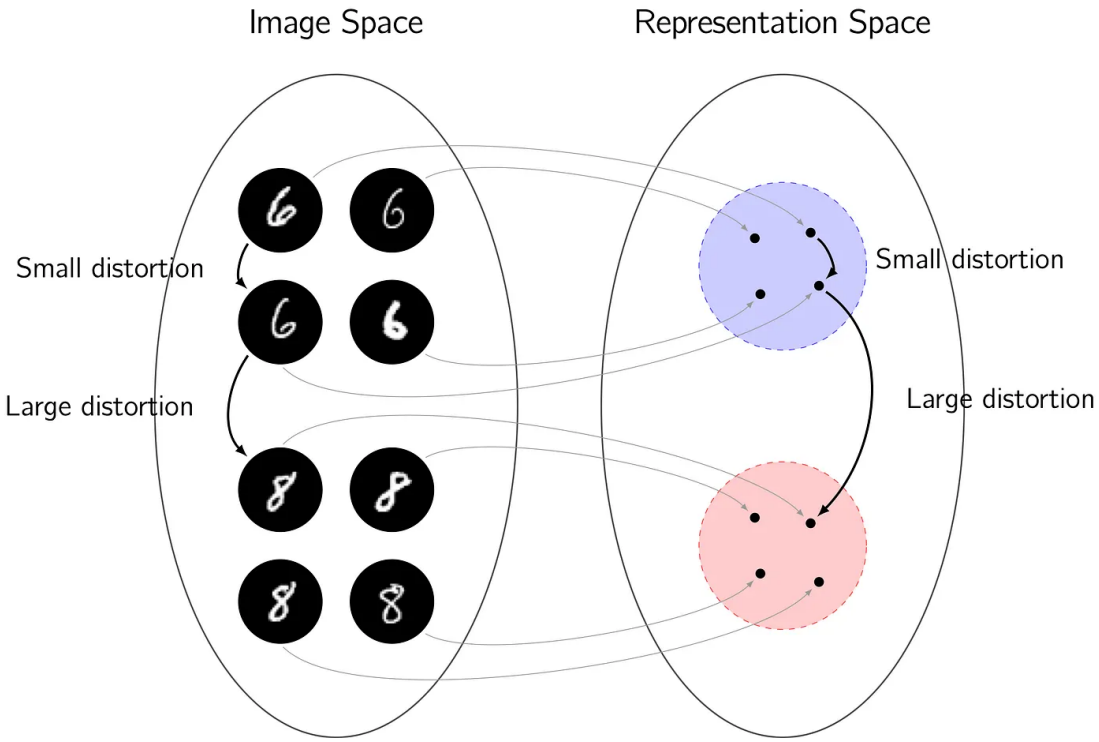


Figure 2. Stability of the mapping to representation space.

A third common geometric prior is to encode a multiscale, hierarchical representation of data. In a data instance, many of the datum are not independent but are correlated in complex ways. Consider an image for example. Each image pixel is not independent but rather nearby pixels are often related and very similar. Different notions of "nearby" are also possible depending on content structure. Effective representational spaces can therefore be constructed by capturing the multiscale, hierarchical nature of much data.

Consider a standard 2D image as an example, such as the image of a castle shown below. The illustration below shows a multiscale, hierarchical representation of the image, with a low-resolution version of the original image in the top-left corner and then remaining image content at different resolutions captured in the other panels of the diagram. This provides a much more efficient representation of the underlying image and, in fact, is the technology powering JPEG-2000 image compression. Similar multiscale, hierarchical representations can be exploited to provide effective

representational spaces for learning.

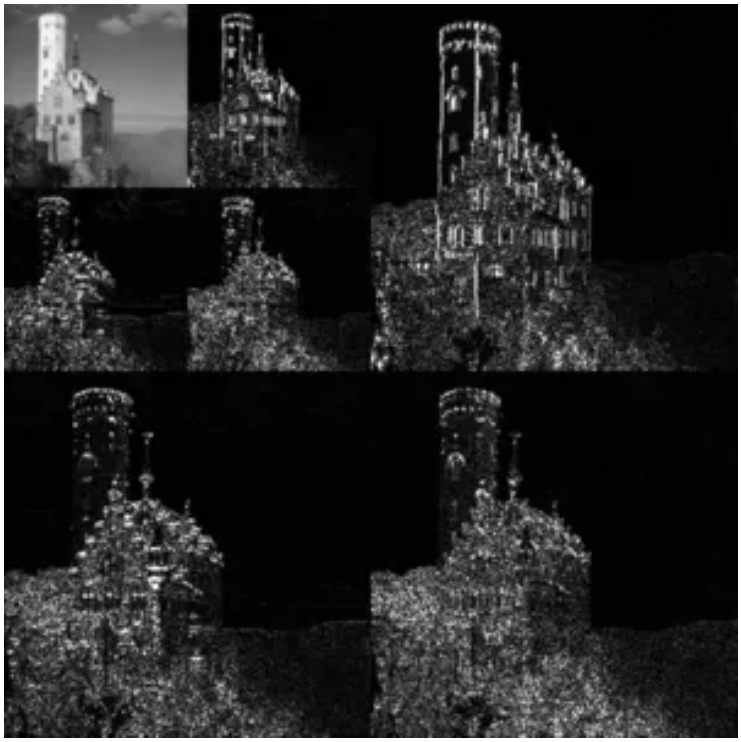


Figure 3. Multiscale hierarchical representation of an image.

We have covered the three main types of geometric priors leveraged in geometric deep learning. While these provide the fundamental underlying concepts of geometric learning, they can be applied in a number of different settings.

Categories of Geometric Deep Learning

In Bronstein's recent book [1], geometric deep learning is classified into four fundamental categories, as illustrated in the diagram below.

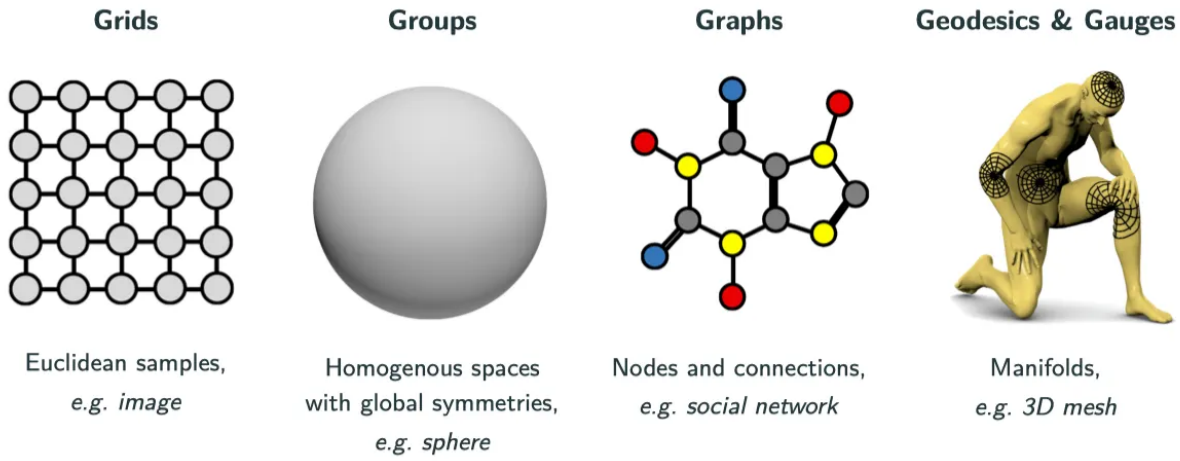


Figure 4. Categories of geometric deep learning: grids, groups, graphs, geodesics and gauges.

Bronstein talks of the 5Gs (extending the 4G categorisation first introduced by Max Welling [1]): grids; groups; graphs; and geodesics and gauges. Since these final two Gs are closely related we consider just four different categories, i.e. 4Gs.

The grid category captures regularly sampled, or gridded, data such as 2D images. These data would perhaps typically be the purveyance of classical deep learning. However, it is also possible to interpret many of the classical deep learning models in a geometric perspective (such as CNNs an their translational equivariance, as discussed above).

The group category covers homogenous spaces with global symmetries. The canonical example of this category is the sphere (covered in greater detail in our previous article [3]). Spherical data arise in myrad applications, not only when data is acquired directly on the sphere (such as over the Earth or by 360° cameras that capture panoramic photos and videos), but also when considering spherical symmetries (such as in molecular chemistry or magnetic resonance imaging). While the sphere is the most common group setting, other groups and their corresponding symmetries can also be considered.

The graph category covers data that may be represented by a computational graph, with nodes and edges. Networks are well-suited to such representations, hence graph deep learning has found wide application in the study of social networks. The graph approach to geometric deep learning provides great flexibility since much data can be represented by a graph. However, this flexibility can come with a loss in specificity and the advantages that affords. For example, the group setting can often be considered with a graph approach but in this case one loses the underlying knowledge of the group, which can otherwise be leveraged.

The final geodesics and gauges category involves deep learning on more complex shapes, such as more general maniolds and 3D meshes. Such approaches can be of great use in computer vision and graphics, for example, where one can perform deep learning with 3D models and their deformations.

Building Blocks

While there are a number of different categories of geometric deep learning, as described above, and different types of geometric priors than can be exploited, all approaches to geometric deep learning essentially adopt different incarnations of the following fundamental underlying building blocks.

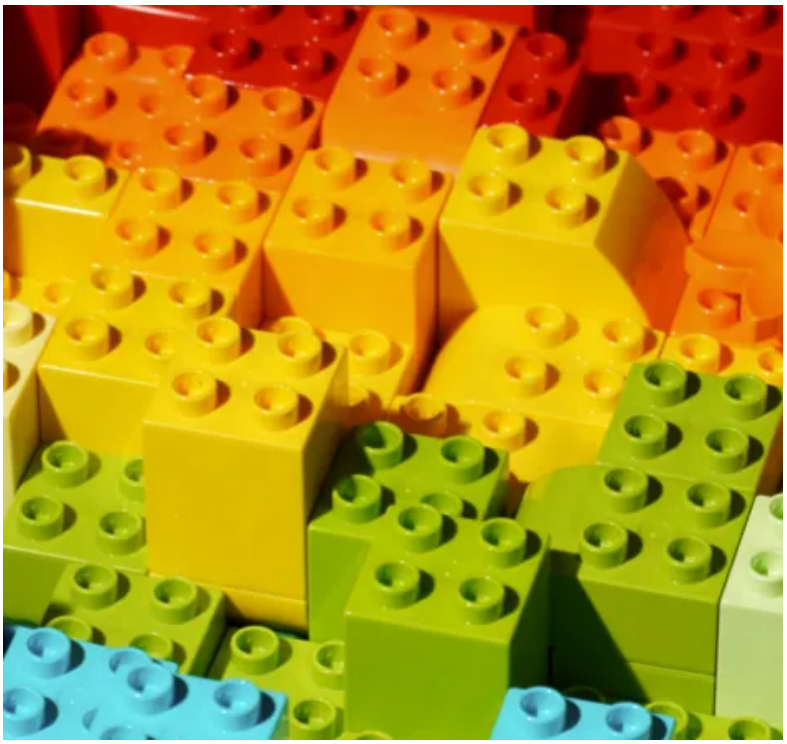


Figure 5. Building blocks of geometric deep learning.

Deep learning architectures are typically composed of a number of layers, that are combined together to form the overall model architecture. Often combinations of layers are then repeated. Geometric deep learning models typically include the following types of layers.

1. Linear equivariant layers: The core component of geometric deep learning models is linear layers, such as convolutions, that are equivariant to some symmetry transformation. The linear transform itself needs to be constructed for the geometric category considered, e.g. a convolution on the sphere and graph are difficult, although there are often many analogies.

2. Non-linear equivariant layers: To ensure deep learning models have sufficient representational power, they must exhibit non-linearity (otherwise they could only represent simple linear mappings). Non-linear layers must be introduced to achieve this, while also preserving equivariance. The canonical way to introduce non-linearity in an equivariant manner it to do so via pointwise non-linear activation functions (e.g. ReLUs), although other forms of non-linearity tailored specifically to the underlying geometry are sometimes considered [3].

3. Local averaging: Most geometric deep learning models also include a form of local averaging, such as max pooling layers in CNNs. Such operations impose local invariances at certain scales, ensuring stability and leading to multi-scale, hierarchical representations by stacking multiple blocks of layers.

4. Global averaging: To impose global invariances in geometric deep learning models, global averging layers are often employed, such as global pooling layers in CNNs.

The canonical example of a geometric deep learning model is a tranditional CNN for 2D planar images. While many may consider this as a classical deep learning model, it can be interpreted in a geometric perspective. Indeed, one of the key reasons CNNs have been so successful is due to the geometric properties encoded in their architecture. The following diagram outlines a typical CNN architecture, where it is clear many of the geometric deep learning layers discussed above are included, with blocks of layers repeated to provide a hierarchical, multiscale representational space.

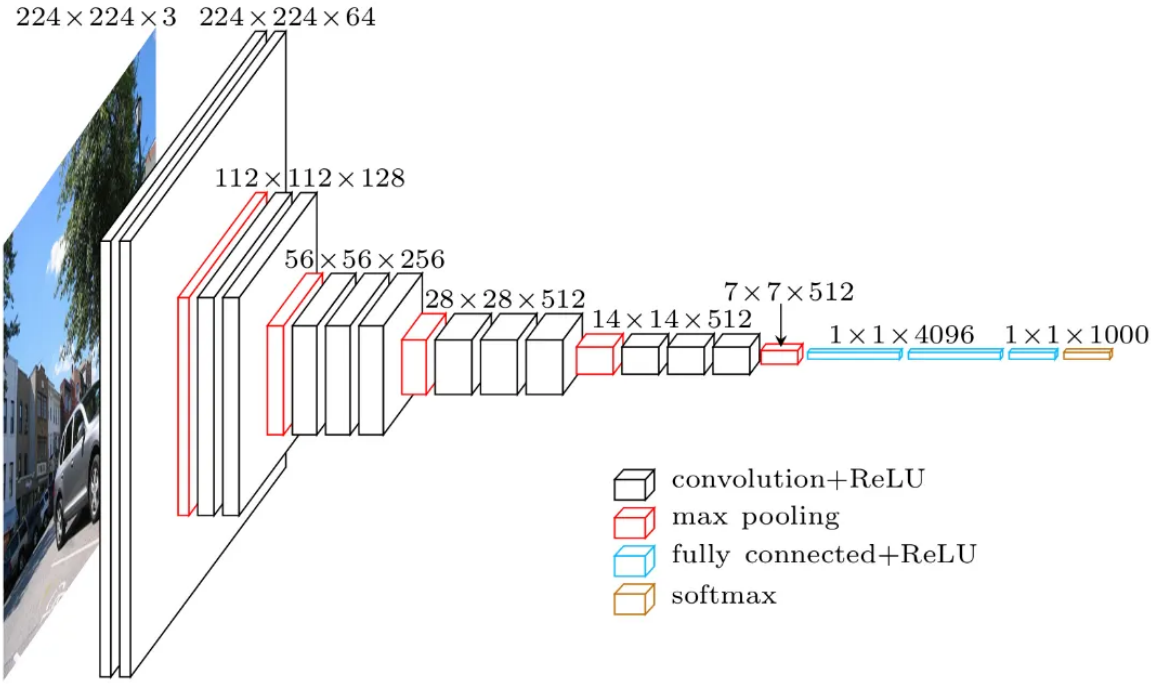


Figure 6. VGG-16 convolutional neural network architecture.

Future perspectives

Deep learning is now commonplace for standard types of data, such as structured, sequential and image data. However, to exend the application of deep learning to other more complex — geometric — datasets, the geometry of such data must be encoded in deep learning models, giving rise to the field of geometric deep learning.

Geometric deep learning is a topical and rapidly evolving field, where much progress has been made. However, many unsolved questions remain, not only in models themselves but also around scalability and practical application. We will address these issuses in upcoming articles, showing how solving such issues is critical to unlocking the remarkable potential of deep learning for a host of new applications.

References

[1] Bronstein, Bruna, Cohen, Velickovic, Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021), arXix:2104.13478

[2] McEwen, Wallis, Mavor-Parker, Scattering Networks on the Sphere for Scalable and Rotationally Equivariant Spherical CNNs, ICLR (2022), arXiv:2102.02828

[3] Cobb, Wallis, Mavor-Parker, Marignier, Price, d'Avezac, McEwen, Efficient Generalised Spherical CNNs, ICLR (2021), arXiv:2010.11661